Rajalakshmi Engineering College

Name: Mohamed Ismail Fawaz K P
Email: 240701324@rajalakshmi.edu.in
Roll no: 240701324
Phone: 9791737785
Branch: REC
Department: CSE - Section 9
Batch: 2028
Degree: B.E - CSE

Scan to verify results

2024_28_III_OOPS Using Java Lab

REC_2028_OOPS using Java_Week 10_MCQ

Attempt : 1
Total Mark : 15
Marks Obtained : 15

Section 1 : MCQ

1. What will be the output of the following code?

```
import java.util.*;
class Main {
   public static void main(String[] args) {
      HashMap<String, Integer> map = new HashMap<>();
      map.put("A", 1);
      map.put("B", 2);
      map.put("C", 3);
      System.out.println(map.containsKey("B"));
   }
}
```

Answer

true

Status : Correct ***Marks : 1/1***

2. What will happen if you add a null element to a TreeSet?

Answer

An exception occurs

Status : Correct ***Marks : 1/1***

3. Which statement is true about HashSet and TreeSet?

Answer

TreeSet provides sorted elements

Status : Correct ***Marks : 1/1***

4. Which of the following is true about HashMap?

Answer

It is not synchronized

Status : Correct ***Marks : 1/1***

5. Which of the following allows null keys in Java?

Answer

HashMap

Status : Correct ***Marks : 1/1***

6. What happens if two keys have the same hash code in a HashMap?

Answer

A linked list is used to store values with the same hash

Status : Correct ***Marks : 1/1***

7. Which method retrieves the lowest key in a TreeMap?

Answer

firstKey()

Status : Correct ***Marks : 1/1***

8. Which of the following is true about TreeMap?

Answer

It maintains natural ordering

Status : Correct ***Marks : 1/1***

9. How does HashSet check for duplicate elements?

Answer

Using equals() and hashCode()

Status : Correct ***Marks : 1/1***

10. What is the time complexity of retrieving an element from a HashSet?

Answer

O(1)

Status : Correct ***Marks : 1/1***

11. What will be the output of the following code?

```
import java.util.*;
class Main {
    public static void main(String[] args) {
        HashMap<String, String> map = new HashMap<>();
        map.put("A", "Apple");
        map.put("B", "Banana");
        map.put("C", "Cherry");
```

```
        map.replace("B", "Blueberry");
        System.out.println(map);
    }
}
```

Answer

{A=Apple, B=Blueberry, C=Cherry}

Status : Correct ***Marks : 1/1***

12. What happens when you add duplicate elements to a HashSet?

Answer

The duplicate is ignored

Status : Correct ***Marks : 1/1***

13. Which method removes all elements from a Set?

Answer

clear()

Status : Correct ***Marks : 1/1***

14. What will be the output of the following code?

```
import java.util.*;
class Main {
    public static void main(String[] args) {
        HashMap<String, Integer> map = new HashMap<>();
        map.put("X", 10);
        map.put("Y", 20);
        map.put("Z", 30);
        map.remove("Y");
        System.out.println(map);
    }
}
```

Answer

{X=10, Z=30}

Status : Correct ***Marks : 1/1***

15. What will happen if you add elements in descending order in a TreeSet?

Answer

They are sorted in ascending order

Status : Correct ***Marks : 1/1***

Rajalakshmi Engineering College

Name: Mohamed Ismail Fawaz K P
Email: 240701324@rajalakshmi.edu.in
Roll no: 240701324
Phone: 9791737785
Branch: REC
Department: CSE - Section 9
Batch: 2028
Degree: B.E - CSE

Scan to verify results

2024_28_III_OOPS Using Java Lab

2028_REC_OOPS using Java_Week 10_Q1

Attempt : 1
Total Mark : 10
Marks Obtained : 10

Section 1 : COD

1. Problem Statement

A city traffic management system needs to track vehicles entering a toll booth. Each vehicle is uniquely identified by its registration number. The system should allow adding vehicles to a record, ensuring that no duplicate registration numbers exist. The vehicles should be stored in a HashSet, which does not guarantee any specific order.

Your task is to implement a program using a HashSet that allows adding vehicle details and displaying the records.

Input Format

The first line of input contains an integer N - the number of vehicles.

The next N lines contain details of each vehicle in the format: "RegNumber

OwnerName VehicleType"

1. RegNumber (String) - A unique registration number (Alphanumeric).
2. OwnerName (String) - The name of the vehicle owner.
3. VehicleType (String, Car, Bike, or Truck) - The type of vehicle.

If a vehicle with the same registration number is already present, ignore the duplicate entry.

Output Format

The output prints the unique vehicle records in any order (since HashSet does not maintain order).

Output format: "RegNumber OwnerName VehicleType"

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 5
KA01AB1234 John Car
MH02CD5678 Alice Bike
DL03EF9012 Bob Truck
TN04GH3456 Mike Car
KA01AB1234 John Car
Output: TN04GH3456 Mike Car
KA01AB1234 John Car
MH02CD5678 Alice Bike
DL03EF9012 Bob Truck

Answer

```
// You are using Java
import java.util.*;
class main{
   public static void main(String args[]){
      Scanner sc=new Scanner(System.in);
      HashSet<String> vn=new HashSet<>();
      int n=sc.nextInt();
      sc.nextLine();
      for(int i=0;i<n;i++){
```

```
            String h=sc.nextLine();
            vn.add(h);
        }
        for(String a: vn){
            System.out.println(a);
        }
    }
}
```

Status : Correct **Marks : 10/10**

Rajalakshmi Engineering College

Name: Mohamed Ismail Fawaz K P
Email: 240701324@rajalakshmi.edu.in
Roll no: 240701324
Phone: 9791737785
Branch: REC
Department: CSE - Section 9
Batch: 2028
Degree: B.E - CSE

Scan to verify results

2024_28_III_OOPS Using Java Lab

2028_REC_OOPS using Java_Week 10_Q2

Attempt : 1
Total Mark : 10
Marks Obtained : 10

Section 1 : COD

1. Problem Statement

John is organizing a fruit festival, and the quantities of various fruits are stored in a HashMap where fruit names are keys and quantities are values.

Help him develop a program to find the total quantity of fruits for the festival by summing up the values in the HashMap.

Input Format

The input consists of fruit quantities in the format 'fruitName:quantity', where fruitName is the name of the fruit(a string), and quantity is a double value representing the quantity.

The input is terminated by entering "done".

Output Format

The output prints a double value, representing the sum of values in the HashMap, rounded off to two decimal places.

If the value is not numeric, print "Invalid input".

If any special characters other than ':' are entered, print "Invalid format".

Refer to the sample output for formatting specifications.

Sample Test Case

Input: Banana:15.2
Orange:56.3
Mango:47.3
done

Output: 118.80

Answer

```
// You are using Java
import java.util.*;

class main{
   public static void main(String args[]){
      Scanner sc=new Scanner(System.in);
      HashMap<String,Double> map=new HashMap<>();
      while(true){
         String a=sc.nextLine().trim();
         if (a.equals("done")){
            break;
         }
         if(!a.contains(":")|| a.indexOf(":")!=a.lastIndexOf(":")){
            System.out.println("Invalid format");
            return;
         }
         String []b=a.split(":");
         if (b.length !=2){
            System.out.println("Invalid format");
            return;
         }
         String n=b[0];
```

```
            String q=b[1];
            if (!n.matches("[A-Za-z]+")){
                System.out.println("Invalid format");
                return;
            }
            double qt;
            try{
                qt=Double.parseDouble(q);
            }
            catch (Exception e){
                System.out.println("Invalid input");
                return;
            }
            map.put(n,qt);
        }
        double sum=0;
        for(double p:map.values()){
            sum+=p;
        }
        System.out.printf("%.2f",sum);
    }
}
```

Status : Correct **Marks : 10/10**

Rajalakshmi Engineering College

Name: Mohamed Ismail Fawaz K P
Email: 240701324@rajalakshmi.edu.in
Roll no: 240701324
Phone: 9791737785
Branch: REC
Department: CSE - Section 9
Batch: 2028
Degree: B.E - CSE

Scan to verify results

2024_28_III_OOPS Using Java Lab

2028_REC_OOPS using Java_Week 10_Q3

Attempt : 1
Total Mark : 10
Marks Obtained : 10

Section 1 : COD

1. Problem Statement

Priya is analyzing encrypted messages in a research project. She wants to analyze the frequency of each character in a given paragraph. The characters should be stored in a TreeMap so that the output is sorted in ascending order of characters automatically.

You are required to build a Java program that:

Uses a TreeMap<Character, Integer> to count how many times each character appears in the message.Ignores spaces and considers only alphabets (case-sensitive).Outputs the frequencies of characters in sorted order.

You must use a TreeMap in the class named MessageAnalyzer.

Input Format

The first line of input contains an integer n, the number of lines in the message.

The next n lines each contain a string (the encrypted message line).

Output Format

The first line of output prints: "Character Frequency:"

Then print each character and its frequency in the format: "<character>: <count>"

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 2
Hello World
Java
Output: Character Frequency:
H: 1
J: 1
W: 1
a: 2
d: 1
e: 1
l: 3
o: 2
r: 1
v: 1

Answer

```
// You are using Java
import java.util.*;

class MessageAnalyzer {

    public void analyzeMessage(List<String> lines) {
        TreeMap<Character, Integer> map = new TreeMap<>();

        for (String line : lines) {
            for (char ch : line.toCharArray()) {
                if (ch == ' ') continue;
```

```
            if (Character.isLetter(ch)) {
                map.put(ch, map.getOrDefault(ch, 0) + 1);
            }
        }
    }

    System.out.println("Character Frequency:");
    for (Map.Entry<Character, Integer> e : map.entrySet()) {
        System.out.println(e.getKey() + ": " + e.getValue());
    }
  }
}

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        int n = Integer.parseInt(sc.nextLine());
        List<String> inputLines = new ArrayList<>();

        for (int i = 0; i < n; i++) {
            inputLines.add(sc.nextLine());
        }

        MessageAnalyzer analyzer = new MessageAnalyzer();
        analyzer.analyzeMessage(inputLines);
    }
}
```

Status : Correct ***Marks : 10/10***

Rajalakshmi Engineering College

Name: Mohamed Ismail Fawaz K P
Email: 240701324@rajalakshmi.edu.in
Roll no: 240701324
Phone: 9791737785
Branch: REC
Department: CSE - Section 9
Batch: 2028
Degree: B.E - CSE

Scan to verify results

2024_28_III_OOPS Using Java Lab

2028_REC_OOPS using Java_Week 10_Q4

Attempt : 1
Total Mark : 10
Marks Obtained : 10

Section 1 : COD

1. Problem Statement

In a ticket reservation system, you store the available seat numbers in a TreeSet. Users input their desired seat number, and the program checks whether the chosen seat is available.

Using a TreeSet ensures quick and efficient verification of seat availability, ensuring a smooth and organized ticket booking process.

Input Format

The first line of input contains a single integer n, representing the number of available seats.

The second line contains n space-separated integers, representing the available seat numbers.

The third line contains an integer m, representing the seat number that needs to be searched.

Output Format

The output displays "[m] is present!" if the given seat is available. Otherwise, it displays "[m] is not present!"

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 4
2 4 5 6
5
Output: 5 is present!

Answer

```
// You are using Java
import java.util.*;

class main{
    public static void main(String args[]){
        Scanner sc=new Scanner(System.in);
        int n=sc.nextInt();
        sc.nextLine();
        TreeSet<Integer> tic=new TreeSet<>();
        for(int i=0;i<n;i++){
            int a=sc.nextInt();
            tic.add(a);
        }
        int c=sc.nextInt();
        if (tic.contains(c)){
            System.out.printf("%d is present!",c);
        }
        else
        System.out.println(c+" is not present!");
    }
}
```

Status : Correct ***Marks : 10/10***

Rajalakshmi Engineering College

Name: Mohamed Ismail Fawaz K P
Email: 240701324@rajalakshmi.edu.in
Roll no: 240701324
Phone: 9791737785
Branch: REC
Department: CSE - Section 9
Batch: 2028
Degree: B.E - CSE

Scan to verify results

2024_28_III_OOPS Using Java Lab

REC_2028_OOPS using Java_Week 10_PAH

Attempt : 1
Total Mark : 30
Marks Obtained : 30

Section 1 : Coding

1. Problem Statement

Sarah is working on a spam detection system that analyzes incoming messages for unique patterns. Spammers often use repetitive character sequences, making it important to identify the first non-repeating character in a message.

Given a string, Sarah needs to determine the first character that appears only once. If all characters repeat, the system should return -1.

She decides to use a HashMap to efficiently track character frequencies and find the solution.

Input Format

The first line contains an integer N representing , the length of the string.

The second line contains a string of N lowercase English letters (a-z).

Output Format

The output prints a character representing the first non-repeating character. If none exist, print -1.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 10
abacabadac
Output: d

Answer

```
// You are using Java
import java.util.*;
class main{
   public static void main(String args[]){
      Scanner sc=new Scanner(System.in);
      int n=Integer.parseInt(sc.nextLine().trim());
      HashMap<Character,Integer> map=new HashMap<>();
      String a=sc.nextLine();
      for(char b:a.toCharArray()){
         map.put(b,map.getOrDefault(b,0)+1);
      }
      for(char b:a.toCharArray()){
         if (map.get(b)==1){
            System.out.println(b);
            return;
         }
      }
      System.out.println("-1");
   }
}
```

Status : Correct ***Marks : 10/10***

2. Problem Statement

A university maintains a list of student records and wants to store them in a sorted manner based on their GPA. If two students have the same GPA, they should be further sorted by their name in lexicographical order. Implement a program that uses a TreeSet to store student records and ensures unique student IDs.

Input Format

The first line contains an integer N - the number of students.

The next N lines contain details of each student in the format: "StudentID Name GPA"

- StudentID (Integer) - A unique identifier.
- Name (String) - The student's name (can contain spaces).
- GPA (Double) - The Grade Point Average.

Output Format

The output prints the list of students in ascending order of GPA.

If two students have the same GPA, sort them by name.

Print details in the format: "StudentID Name GPA" in the output, GPA is rounded to two decimal places.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 5
101 John 8.5
102 Alice 9.1
103 Bob 8.5
104 Zoe 7.3
105 Charlie 9.1
Output: 104 Zoe 7.30
103 Bob 8.50
101 John 8.50

102 Alice 9.10
105 Charlie 9.10

Answer

```
// You are using Java
import java.util.*;

class Student implements Comparable<Student> {
    int id;
    String name;
    double gpa;

    Student(int id, String name, double gpa) {
        this.id = id;
        this.name = name;
        this.gpa = gpa;
    }

    // Sorting logic: GPA asc    Name asc    ID asc (to avoid collision)
    @Override
    public int compareTo(Student other) {
        if (this.gpa != other.gpa) {
            return Double.compare(this.gpa, other.gpa);  // GPA ascending
        }
        int nameCompare = this.name.compareTo(other.name);
        if (nameCompare != 0) {
            return nameCompare;  // Name ascending
        }
        return Integer.compare(this.id, other.id);  // Final tie breaker
    }

    // Ensuring unique ID using equals + hashCode
    @Override
    public int hashCode() {
        return Integer.hashCode(id);
    }

    @Override
    public boolean equals(Object obj) {
        if (!(obj instanceof Student)) return false;
        return this.id == ((Student)obj).id;
    }
```

```
}
public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        int n = Integer.parseInt(sc.nextLine());

        TreeSet<Student> set = new TreeSet<>();

        for (int i = 0; i < n; i++) {
            String line = sc.nextLine().trim();

            String[] parts = line.split(" ");

            int id = Integer.parseInt(parts[0]);

            // Name may contain spaces    extract from index 1 to second last index
            double gpa = Double.parseDouble(parts[parts.length - 1]);

            StringBuilder sb = new StringBuilder();
            for (int j = 1; j < parts.length - 1; j++) {
                sb.append(parts[j]);
                if (j != parts.length - 2) sb.append(" ");
            }

            String name = sb.toString();

            set.add(new Student(id, name, gpa));
        }

        for (Student s : set) {
            System.out.printf("%d %s %.2f\n", s.id, s.name, s.gpa);
        }
    }
}
```

Status : Correct ***Marks : 10/10***

3. Problem Statement

Riya is building a calendar event scheduler where each event is stored in chronological order using a TreeMap. The key represents the event time in 24-hour format (HH:MM), and the value is the event description.

She wants the system to:

Automatically sort events by time.Avoid duplicate time entries — if a duplicate time is entered, ignore the new entry.Print all scheduled events in order.

Implement this logic using a class named EventManager.

Input Format

The first line of the input contains an integer n, representing the number of events.

The next n lines each contain a string in the format: "HH:MM Description"

(Example: 09:00 TeamMeeting).

Output Format

The first line of the output prints "Scheduled Events:"

The next k lines print each event in the format: "HH:MM - Description"

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 5
09:00 TeamMeeting
13:30 LunchBreak
11:00 ProjectUpdate
09:00 Standup
15:00 ClientCall
Output: Scheduled Events:
09:00 - TeamMeeting
11:00 - ProjectUpdate
13:30 - LunchBreak

15:00 - ClientCall

Answer

```
// You are using Java
import java.util.*;

class EventManager {
    private TreeMap<String, String> events = new TreeMap<>();

    public void addEvent(String time, String description) {
        if (!events.containsKey(time)) {
            events.put(time, description);
        }
    }

    public void printEvents() {
        System.out.println("Scheduled Events:");
        for (Map.Entry<String, String> e : events.entrySet()) {
            System.out.println(e.getKey() + " - " + e.getValue());
        }
    }
}

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = Integer.parseInt(sc.nextLine());

        EventManager manager = new EventManager();

        for (int i = 0; i < n; i++) {
            String line = sc.nextLine().trim();
            String[] parts = line.split(" ");
            manager.addEvent(parts[0], parts[1]);
        }

        manager.printEvents();
    }
}
```

Status : Correct ***Marks : 10/10***

Rajalakshmi Engineering College

Name: Mohamed Ismail Fawaz K P
Email: 240701324@rajalakshmi.edu.in
Roll no: 240701324
Phone: 9791737785
Branch: REC
Department: CSE - Section 9
Batch: 2028
Degree: B.E - CSE

Scan to verify results

2024_28_III_OOPS Using Java Lab

REC_2028_OOPS using Java_Week 10_CY

Attempt : 1
Total Mark : 40
Marks Obtained : 40

Section 1 : COD

1. Problem Statement

David is managing an employee database where each employee has a unique ID, name, and department. He wants to ensure that duplicate employee IDs are not added to the system. Implement a Java program that allows adding employees to the system, displaying all employees, and checking if an employee exists based on the given ID.

Implement a class EmployeeDatabase that contains a HashSet to store employee records. The Employee class should be a user-defined object containing employee details. The main class should handle user operations and interact with the EmployeeDatabase class.

Input Format

The first line contains an integer n representing the number of employees to be added.

The next n lines follow, each containing:

1. An integer employee_id
2. A string name
3. A string department

The next line contains an integer m representing the number of queries.

The next m lines follow, each containing an employee ID to check for existence.

Output Format

The output prints a list of all employees added in the format:

"ID: <employee_id>, Name: <name>, Department: <department>"

For each query, output "Employee exists" if the ID is found, otherwise "Employee not found".

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 3
101 John IT
102 Alice HR
103 Bob Finance
2
101
104

Output: ID: 101, Name: John, Department: IT
ID: 102, Name: Alice, Department: HR
ID: 103, Name: Bob, Department: Finance
Employee exists
Employee not found

Answer

```
import java.util.*;

class Employee {
```

```
    int id;
    String name;
    String department;

    Employee(int id, String name, String department) {
        this.id = id;
        this.name = name;
        this.department = department;
    }

    @Override
    public boolean equals(Object o) {
        if (this == o) return true;
        if (!(o instanceof Employee)) return false;
        Employee e = (Employee) o;
        return this.id == e.id;
    }

    @Override
    public int hashCode() {
        return Objects.hash(id);
    }
}

class EmployeeDatabase {

    private HashSet<Employee> employees = new HashSet<>();

    public void addEmployee(int id, String name, String department) {
        employees.add(new Employee(id, name, department));
    }

    public void displayEmployees() {
        List<Employee> list = new ArrayList<>(employees);
        list.sort(Comparator.comparingInt(e -> e.id));

        for (Employee e : list) {
            System.out.println("ID: " + e.id + ", Name: " + e.name + ", Department: " +
e.department);
        }
    }
```

```
    public boolean checkEmployee(int id) {
        for (Employee e : employees) {
            if (e.id == id) return true;
        }
        return false;
    }
}



class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        EmployeeDatabase db = new EmployeeDatabase();
        int n = sc.nextInt();
        for (int i = 0; i < n; i++) {
            int id = sc.nextInt();
            String name = sc.next();
            String department = sc.next();
            db.addEmployee(id, name, department);
        }
        db.displayEmployees();
        int m = sc.nextInt();
        for (int i = 0; i < m; i++) {
            int id = sc.nextInt();
            if (db.checkEmployee(id))
                System.out.println("Employee exists");
            else
                System.out.println("Employee not found");
        }
        sc.close();
    }
}
```

Status : Correct ***Marks : 10/10***

2. Problem Statement

Bob wants to develop a score-tracking application for a gaming

tournament. Each player's score is stored in a HashMap with the player's name as the key and the score as the value.

Write a program to assist Bob that takes user input to enter player scores, calculates the maximum score from the HashMap, and prints the player with the highest score.

Input Format

The input consists of strings representing player details in the format "playerName:score".

The input is terminated by entering "done".

Output Format

The output displays a string, representing the player's name who scored the maximum.

If the value is not numeric, print "Invalid input".

If any special characters other than ':' are given, print "Invalid format".

Refer to the sample output for formatting specifications.

Sample Test Case

Input: Alice:15
Bob:56
done
Output: Bob

Answer

```
import java.util.*;

class ScoreTracker {

    HashMap<String, Integer> scoreMap = new HashMap<>();

    public boolean processInput(String input) {
```

```
        if (!input.contains(":") || input.indexOf(':') != input.lastIndexOf(':')) {
            System.out.println("Invalid format");
            return false;
        }

        String[] parts = input.split(":");
        if (parts.length != 2) {
            System.out.println("Invalid format");
            return false;
        }

        String name = parts[0];
        String scoreStr = parts[1];

        if (!name.matches("[A-Za-z]+")) {
            System.out.println("Invalid format");
            return false;
        }

        if (!scoreStr.matches("\\d+")) {
            System.out.println("Invalid input");
            return false;
        }

        int score = Integer.parseInt(scoreStr);

        scoreMap.put(name, score);
        return true;
    }

    public String findTopPlayer() {
        String topPlayer = "";
        int maxScore = -1;

        for (Map.Entry<String, Integer> entry : scoreMap.entrySet()) {
            if (entry.getValue() > maxScore) {
                maxScore = entry.getValue();
                topPlayer = entry.getKey();
            }
        }

        return topPlayer;
```

```
    }
}
public class Main {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        ScoreTracker tracker = new ScoreTracker();
        boolean validInput = true;

        while (true) {
            String input = scanner.nextLine();

            if (input.toLowerCase().equals("done")) {
                break;
            }

            if (!tracker.processInput(input)) {
                validInput = false;
                break;
            }
        }

        if (validInput && !tracker.scoreMap.isEmpty()) {
            System.out.println(tracker.findTopPlayer());
        }

        scanner.close();
    }
}
```

Status : Correct **Marks : 10/10**

3. Problem Statement

A college professor wants to keep track of students who attend classes. Each student has a unique roll number and their attendance count increases every time they attend a class. The system should allow adding a student, marking their attendance, and displaying all students with their total attendance.

Your task is to implement a Java program using TreeSet to maintain

students in sorted order of roll numbers and track their attendance count.

Operations:

A roll_no name Add a student with roll number and name (if not already added).M roll_no Mark attendance for the student with the given roll number (increase their count by 1).D Display all students in ascending order of roll number along with their attendance count.

Input Format

The first line contains an integer N - the number of students.

The next N lines contain one of the following commands:

A roll_no name

M roll_no

D

- A (Add) Adds a new student with a unique roll number and name.
- M (Mark) Increases attendance count for the given roll number.
- D (Display) Prints all students in ascending order of roll number.

Output Format

For D, output prints each student's roll number, name, and attendance count in ascending order of roll number.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 5
A 101 Alice
A 102 Bob
M 101
M 101
D
Output: 101 Alice 2
102 Bob 0

Answer

```
// You are using Java
import java.util.*;

class Student implements Comparable<Student> {
    int rollNo;
    String name;
    int attendance;

    Student(int rollNo, String name) {
        this.rollNo = rollNo;
        this.name = name;
        this.attendance = 0;
    }

    @Override
    public int compareTo(Student s) {
        return Integer.compare(this.rollNo, s.rollNo);
    }

    @Override
    public boolean equals(Object o) {
        if (this == o) return true;
        if (!(o instanceof Student)) return false;
        Student s = (Student) o;
        return this.rollNo == s.rollNo;
    }

    @Override
    public int hashCode() {
        return Objects.hash(rollNo);
    }
}

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int N = sc.nextInt();

        TreeSet<Student> students = new TreeSet<>();

        for (int i = 0; i < N; i++) {
```

```
        String command = sc.next();

        if (command.equals("A")) {
            int roll = sc.nextInt();
            String name = sc.next();

            // Add only if not already present
            students.add(new Student(roll, name));
        }
        else if (command.equals("M")) {
            int roll = sc.nextInt();

            // Find student and increase attendance
            for (Student s : students) {
                if (s.rollNo == roll) {
                    s.attendance++;
                    break;
                }
            }
        }
        else if (command.equals("D")) {
            for (Student s : students) {
                System.out.println(s.rollNo + " " + s.name + " " + s.attendance);
            }
        }
    }

    sc.close();
  }
}
```

Status : Correct **Marks : 10/10**

4. Problem Statement

Tony is an e-learning platform administrator, he oversees the user ratings for various online courses offered in the platform.

To enhance user experience, you should assist him in utilizing a HashMap

to store course ratings given by learners. Regularly, he analyzes this data to identify the highest and lowest-rated courses, enabling targeted improvements and ensuring the quality of the educational content. This process assists in maintaining a competitive and engaging online learning environment for the users.

Input Format

The input consists of a string representing the course name followed by a double value representing the course's rating, in separate lines.

The input is terminated by entering "done".

Output Format

The first line of output prints the string "Highest Rated Course: " followed by the highest-rated course.

The second line prints the string "Lowest Rated Course: " followed by the lowest-rated courses.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: DSA
4.0
OOPS
4.2
C
3.2
done

Output: Highest Rated Course: OOPS
Lowest Rated Course: C

Answer

```
import java.util.HashMap;
import java.util.Map;
import java.util.Scanner;

import java.util.*;
```

```
class CourseAnalyzer {

    public Map<String, String>
identifyHighestAndLowestRatedCourses(Map<String, Double> courseRatings) {

        String highestCourse = "";
        String lowestCourse = "";
        double highestRating = -1.0;
        double lowestRating = Double.MAX_VALUE;

        for (Map.Entry<String, Double> entry : courseRatings.entrySet()) {
            double rating = entry.getValue();
            String course = entry.getKey();

            if (rating > highestRating) {
                highestRating = rating;
                highestCourse = course;
            }

            if (rating < lowestRating) {
                lowestRating = rating;
                lowestCourse = course;
            }
        }

        Map<String, String> result = new HashMap<>();
        result.put("highest", highestCourse);
        result.put("lowest", lowestCourse);

        return result;
    }
}

public class Main {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        Map<String, Double> courseRatings = new HashMap<>();

        while (true) {
            String courseName = scanner.nextLine();
            if (courseName.equalsIgnoreCase("done")) {
                break;
```

```
            }
            double rating = Double.parseDouble(scanner.nextLine().trim());
            courseRatings.put(courseName, rating);
        }

        CourseAnalyzer analyzer = new CourseAnalyzer();
        Map<String, String> result =
analyzer.identifyHighestAndLowestRatedCourses(courseRatings);

        System.out.printf("Highest Rated Course: %s\n", result.get("highest"));
        System.out.printf("Lowest Rated Course: %s", result.get("lowest"));

        scanner.close();
    }
}
```

Status : Correct **Marks : 10/10**